Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

Discuss:

- Response-time, reliability, throughput, battery and security requirements of each device
- Which requirements conflict (for example: fast response and many users on one machine)
- Which requirements are critical to patient safety
- How the lessons from the history of operating systems (batch, time-sharing, personal, mobile) apply to each environment

You are expected to **rank the requirements of each environment and justify the ranking**.

Alternative Concept and Complexity Analysis

If the whole hospital used a single general-purpose operating system on every device, instead of a different type of OS for each environment, what would the solution be? Evaluate the worst-case alarm response time (unbounded scheduling delay versus a guaranteed deadline), the memory needed on each device, and the administration effort.

Evidence Expected

A ranked requirement table for every environment with a justification for each ranking.

TASK 2 – Judge the Type of Operating System for Each Environment **3 Marks**

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

Discuss:

- Mainframe, desktop, multiprocessor, distributed, clustered, real-time (hard and soft) and handheld systems
- Which type best fits the monitors, tablets, workstations, database server, imaging server and partner link
- Symmetric versus asymmetric multiprocessing for the imaging server
- A clustered system with fail-over for the database server

Alternative Concept and Complexity Analysis

If the database server used a single large multiprocessor machine instead of a cluster of smaller machines with fail-over, what would the solution be? Evaluate the availability (a single point of failure versus fail-over within seconds), the cost, and how throughput grows with the number of processors (time) and the memory available (space).

Evidence Expected

A justified choice of operating-system type for each environment, with the rejected alternatives and reasons.

TASK 3 – Evaluate Operating-System Structures **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the possible structures of the operating system.

Discuss:

- Simple (monolithic), layered, microkernel, modular (loadable kernel modules) and hybrid structures
- The effect of each structure on reliability, performance and ease of updating
- Which structure suits the patient monitors and which suits the database server
- How Linux, Windows, Android and a real-time OS such as FreeRTOS or QNX are structured

Alternative Concept and Complexity Analysis

If the bedside monitors used a monolithic kernel instead of a microkernel, what would the solution be? Evaluate the time cost of a service request (one system call versus message passing between servers), the memory taken by the kernel, and the effect of a faulty driver on the whole system.

Evidence Expected

A comparison of structures against the hospital's requirements, with a defended choice for two environments.

TASK 4 – Evaluate the Services an Operating System Must Provide **4 Marks**

Bloom's Level: K5 – Evaluate

Evaluate the services each environment needs.

Discuss:

- User interface, program execution, I/O operations, file systems, communication, error detection, resource allocation, accounting, protection and security
- Which services can be removed from the bedside monitor to make it smaller and faster
- Which services are essential for the database server
- How the OS protects one patient's records from another user's programs

Alternative Concept and Complexity Analysis

If the tablets used a full desktop operating system with every service enabled instead of a reduced handheld OS, what would the solution be? Evaluate the memory and storage used by the extra services (space), the start-up time and the battery life (time).

Evidence Expected

A service table for each environment with justified inclusions and exclusions.

TASK 5 – Examine System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Examine how a program obtains OS services.

Discuss:

- The categories of system calls (process control, file management, device management, information maintenance, communication, protection)
- What happens step by step when a program calls read(): library call, trap, mode switch, kernel work, return
- How parameters are passed to the kernel (registers, a block in memory, the stack)
- How strace shows the system calls made by a program

Alternative Concept and Complexity Analysis

If programs called the C library functions fopen, fread and fwrite instead of the open, read and write system calls directly, what would the solution be? Evaluate the number of system calls for a file of n bytes (about n / 4096 with library buffering versus n / b for a buffer of b bytes) and the extra buffer space used by the library.

Evidence Expected

A step-by-step explanation of a system call with strace evidence from a real program.

TASK 6 – Evaluate the Cost of System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the number of system calls affects performance.

Discuss:

- Why each system call costs much more than an ordinary function call
- How the buffer size changes the number of read() and write() calls when the database copies a file

- How library buffering (fread, fwrite) reduces the number of system calls
- The best buffer size for the hospital's backup program, using your measurements

Complete the Code

Complete TODO 1 below using the read() and write() system calls, and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;
    // TODO 1: Loop: read up to bufsize bytes into buf (count every read call in calls);
    //   stop when read returns 0 (end of file) or -1; write exactly the n bytes that
    //   were read

    close(in);
    close(out);
    return calls;
}

int main(void)
{
    FILE *f = fopen("records.dat", "w");     /* a 10,000-byte test file */
    for (int i = 0; i < 10000; i++) fputc('A' + i % 26, f);
    fclose(f);

    size_t sizes[] = {1, 512, 4096};
    for (int i = 0; i < 3; i++)
        printf("buffer %4zu bytes: %5ld read() calls\n", sizes[i],
               copy_file("records.dat", "copy.dat", sizes[i]));
    unlink("records.dat");
    unlink("copy.dat");
    return 0;
}
```

Expected output:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Run the program under strace -c and compare the number of read() calls reported with the program's output.

Alternative Concept and Complexity Analysis

If the backup program used memory-mapped files (mmap) instead of read() and write() with a buffer, what would the solution be? Evaluate the number of system calls (a few versus about n / b), the copying between kernel and user space, and the memory used for very large files.

Evidence Expected

Measured system-call counts and times for different buffer sizes, with a justified recommendation.

TASK 7 – Evaluate Process Control System Calls — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how a command shell on the database server starts and manages programs.

Discuss:

- What fork(), exec(), wait() and exit() each do
- How a shell runs a command, and how it obtains the command's exit status
- What a zombie and an orphan process are, and how the OS deals with them
- Why fork() followed by exec() is used instead of a single "create process" call

Complete the Code

Complete TODO 1 below using fork(), execlp() and waitpid(), and execute the program on Linux.

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("shell: running job 1\n");
    fflush(stdout);                          /* empty the buffer before fork() */
    // TODO 1: Create a child with fork(). In the child, replace the program with
    //   execlp("expr", "expr", "6", "*", "7", (char *)NULL) and exit(127) if exec
    //   fails. In the parent, wait for the child with waitpid() and print "job 1 exit
    //   status N" using WEXITSTATUS



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3);                  /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

Explain why stdout is flushed before each fork(). Then remove the second fflush(stdout), change _exit(3) to exit(3), redirect the output to a file and explain why "shell: running job 2" now appears twice.

Alternative Concept and Complexity Analysis

If commands were run with posix_spawn() or with threads instead of fork() followed by exec(), what would the solution be? Evaluate the time to start 1,000 commands, the memory copied or shared (copy-on-write), and the isolation between commands.

Evidence Expected

A working program that runs commands like a shell, with process diagrams and a reasoned evaluation.

TASK 8 – Evaluate CPU States and Protection — 3 Marks

Bloom's Level: K5 – Evaluate

Evaluate how the hardware protects the hospital system.

Discuss:

- User mode and kernel mode, privileged instructions and the mode bit
- Interrupts, traps and the timer interrupt
- What would happen to the monitors if any program could disable interrupts
- How the hardware stops one program from reading another program's memory

Alternative Concept and Complexity Analysis

If the monitors used polling (checking the sensor in a loop) instead of interrupts, what would the solution be? Evaluate the worst-case delay before an abnormal reading is noticed (the polling interval versus the interrupt latency) and the CPU time and power wasted by polling.

Evidence Expected

A reasoned explanation of CPU modes, interrupts and protection, applied to the monitors.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems — 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate how data moves between storage levels and devices.

Discuss:

- The levels of the memory hierarchy (registers, cache, main memory, SSD, hard disk, network storage) with typical access times and sizes
- Where MRI images, patient records and monitor readings should be kept
- Programmed I/O, interrupt-driven I/O and DMA, and which suits the MRI scanner
- The effect of caching on the speed of record look-up

Alternative Concept and Complexity Analysis

If the MRI scanner transferred images with interrupt-driven I/O (one interrupt per word) instead of DMA, what would the solution be? Evaluate the number of interrupts and the CPU time for a 500 MB image, and the memory buffers needed in each method.

Evidence Expected

A justified data-placement plan and I/O method for each device, using typical access times.

TASK 10 – Measure and Compare **4 Marks**

Bloom's Level: K5 – Evaluate

Measure the behaviour of your programs on a Linux system.

Discuss:

- The number of system calls made by your copy program for different buffer sizes (strace -c)
- The time spent in user mode and in kernel mode (the time command)
- The time to create 1,000 processes with fork() and to create 1,000 threads
- How the measurements support or contradict your earlier judgements

Alternative Concept and Complexity Analysis

If threads were created instead of processes for the measurements, what would the result be? Evaluate the creation time for 1,000 threads versus 1,000 processes and the memory used by each, and explain the difference using what each one shares.

Evidence Expected

Measured results in tables, compared with expectations, with explained differences.

TASK 11 – Recommend and Defend the Operating-System Plan **4 Marks**

Bloom's Level: K5 – Evaluate

Recommend the complete operating-system plan for the hospital.

Discuss:

- The type and structure of operating system for each environment
- The services, protection and I/O methods each will use
- The limitations and risks of your plan
- Whether the plan remains suitable for the group of three hospitals

Your recommendation must be based on **evidence from your own analysis and measurements**.

Alternative Concept and Complexity Analysis

If the whole system were moved to virtual machines on two large servers, instead of separate physical machines for each environment, what would the solution be? Evaluate the effect on response time for the monitors, the memory and processor overhead of virtualisation, and the effect on reliability.

Evidence Expected

A technically justified plan supported by analysis and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The hospital adds **remote monitoring devices in patients' homes** that send readings over the internet, and moves patient records to a **cloud service** shared with the partner hospitals.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Identify which operating-system types, services and protection mechanisms are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your plan can support home devices and cloud records, considering response time, reliability and security.

Evidence Expected

A technically supported evaluation of the original design.

C. Adaptation or Redesign – 2 Marks

Describe the revised plan (for example: an embedded real-time OS on home devices, a distributed system for shared records).

Evidence Expected

An appropriate design response.

D. Evidence-Based Defence – 2 Marks

Defend your decision using evidence and the trade-offs involved.

Evidence Expected

A well-supported final decision integrating evidence and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3

Task	Bloom's Level	Marks
Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

OPERATING SYSTEMS (CSA04) - ASSIGNMENT 1 : ANSWER SHEET

Name	Monika.S
Register number	1924260182
Institution / course	SIMATS Engineering - CSA04 Operating Systems
GitHub repository name	CSA04-1924260182-Assignment1
GitHub repository link	_____ (paste your link after pushing)
Compiler	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1)
Linux distribution	Ubuntu 24.04.5 LTS, kernel 6.18 (x86_64), page size 4096 B
Compile command	gcc -std=c11 -Wall -Wextra -pthread file.c -o file

Note on measurements. All programs below were compiled and executed on the machine described above (a 1-vCPU Xeon 2.1 GHz virtual machine, 4 GB RAM). Every number marked *measured* comes from that run; your own run will differ in the last digits, so re-run and paste your own screenshots into the repository. strace was not installed on that machine, so the strace commands are given with the output you should expect, not pasted output.

- Repository layout: Task01 ... Task12 folders; Task06 (copy.c), Task07 (shell.c, shell_noflush.c), Task10 (forkthread.c, copy_bench.c); one commit per task; README.md with the table above.

TASK 1 - Requirements of each computing environment (4 marks)

Rank 1 = most important for that environment. Justification follows the table.

Environment	Response time	Reliability	Throughput	Battery	Security	Justification of the ranking
Bedside monitor	1	2	5	3	4	Alarm within 10 ms is a hard deadline: a late alarm = a failed alarm. A crash is next worst (no monitoring). Little data, so throughput is irrelevant; mains + backup cell so battery is third.
Nurse tablet	3	4	5	1	2	Must last a 8-12 h shift. Lost/stolen device holds patient data, so encryption and remote wipe rank 2. Response only needs to be "snappy" (about 100 ms).
Doctor workstation	2	3	4	n/a	1	Many records opened per day: privacy/audit first, then interactive response; can be rebooted, so reliability is third.
Records DB server	4	1	3	n/a	2	"Must never stop": availability first; records are the most sensitive asset (2); throughput of many transactions (3); a few ms latency is acceptable.
Imaging (MRI) server	4	2	1	n/a	3	Heavy parallel number-crunching: throughput first; a failed job can be restarted so reliability 2.
Partner link	4	2	3	n/a	1	Records cross the hospital boundary: authentication, encryption and audit are the main risk; reliability next (shared care).

Conflicting requirements

- Fast response vs many users on one machine: time-sharing gives fairness, not deadlines; one heavy user can delay another. This is why the monitor gets its own real-time OS.
- Throughput vs response: big time slices and batching raise throughput but increase latency (imaging server wants the first, monitor the second).
- Battery vs response/security: keeping the CPU asleep saves power but wakes slowly; full-disk encryption and many background services cost energy on the tablet.
- Security vs availability: applying a security patch normally needs a reboot, which conflicts with a database that must never stop (solved by a cluster with rolling updates).
- Generality vs size: a general-purpose OS has many services (large, slower, more bugs); a small OS is predictable but less flexible.

Requirements critical to patient safety

- Monitor alarm deadline (10 ms) and monitor availability - directly life-critical.
- Integrity and availability of the patient record (wrong or missing allergy data harms patients) - database reliability.
- Confidentiality is a legal/ethical requirement but not immediately life-critical; if it conflicts with alarm timing, alarm timing wins on the monitor.

Lessons from the history of operating systems

Era	Lesson	Where it applies
Batch	Maximise CPU use for long jobs without user interaction; spool and queue.	Imaging server (MRI jobs queued, run to completion), nightly backups.

Era	Lesson	Where it applies
Time-sharing	Short time slices give each user the illusion of a private machine; protection between users is essential.	Doctor workstations, database server (many users/sessions).
Personal / desktop	Responsive GUI for one user; ease of use over raw throughput.	Doctor workstations.
Mobile / handheld	Power management, touch UI, app sandboxing, small memory.	Nurse tablets (and remote home devices later).
Real-time / embedded	Predictable timing and tiny footprint beat fairness.	Bedside monitors.

Alternative: one general-purpose OS on every device

Criterion	Single general-purpose OS everywhere	Different OS per environment (chosen)
Worst-case alarm response	Unbounded: non-preemptible kernel sections, page faults, interrupt storms, priority inversion; typical worst case tens to hundreds of ms under load. O(unbounded) - the 10 ms deadline cannot be guaranteed.	RTOS with fixed-priority preemptive scheduling: bounded, worst case = interrupt latency + context switch + task time, typically well under 1 ms. O(1) bounded.
Memory on each device	Hundreds of MB to GB (kernel + services) even on a monitor that needs about 1 MB.	Monitor RTOS: tens of KB to a few MB; tablet mobile OS: 1-4 GB; servers: as needed.
Administration effort	One image, one patch process, one skill set (cheaper for 500 + 120 + 80 devices) but every patch/reboot hits every device type, incl. the DB and monitors.	About 4-5 images and skill sets, but each is small, patched independently, and a failure stays inside its environment.

TASK 2 - Type of operating system for each environment (3 marks)

Environment	Best-fit type	Rejected alternatives and reasons
Bedside monitors	Hard real-time OS (e.g. QNX / FreeRTOS on embedded hardware)	Soft real-time/Linux: occasional deadline misses unacceptable. Desktop OS: unbounded latency, too large.
Nurse tablets	Handheld/mobile OS (Android-class) with MDM	Desktop OS: large memory, high power, no touch/power management. Real-time OS: unnecessary and has no app ecosystem.
Doctor workstations	Desktop (time-sharing, GUI) OS - Windows/Linux desktop	Mobile OS: too limited for large records and imaging viewers. RTOS: no rich GUI.
Database server	Server OS (Linux) in a clustered system with fail-over (2-3 nodes)	Single big machine: single point of failure. Desktop OS: weak server tuning. Mainframe: costly (acceptable only for very large groups).
Imaging server	Multiprocessor server OS using SMP (Linux), with a job queue	AMP: wastes a master CPU and is a bottleneck. Cluster: only if one machine becomes too small (later).
Partner link	Distributed system layer on a server OS (TLS/VPN, replication of records)	Direct DB access from partners: insecure and fragile. Ad-hoc file copying: stale, unaudited data.

- **Hard vs soft real-time:** hard = missing a deadline is a failure (alarm 10 ms); soft = occasional misses only degrade quality (video display, nurse-tablet alerts).
- **SMP vs AMP for the imaging server:** SMP: every CPU runs the same kernel and picks work from a shared ready queue, so load balances automatically and no CPU is a bottleneck; costs: lock contention in the kernel and cache coherence traffic. AMP: a master CPU assigns work to slaves; simple, but the master saturates and its failure stops everything. Choose **SMP**.
- **Clustered DB with fail-over:** two or more nodes share storage (or replicate); a heartbeat detects node failure and the standby takes over within seconds, and nodes can be patched one at a time (rolling update), meeting "must never stop".

Alternative: single large multiprocessor machine for the database

Criterion	Single large SMP machine	Cluster with fail-over (chosen)
Availability	Single point of failure. With MTBF 10,000 h, MTTR 4 h: A = 10000/10004 = 99.96 % (about 3.5 h down per year).	Both nodes must fail together: about $1-(0.0004)^2$; outage is only the fail-over time (about 20 s per active-node failure, about 0.88 failures/yr = about 18 s/yr): about 99.9999 %.
Cost	One very expensive machine (cost grows faster than linearly with CPUs), but one licence and simple.	Two or more cheaper nodes + shared storage/replication software; total often lower, more complex.
Throughput vs processors (time)	Amdahl: S(N) = 1 / ((1-p) + p/N). For p = 0.95: N = 8 gives 5.9, N = 16 gives 9.1, N = 64 gives 15.4, limit 20. Kernel lock contention flattens the curve.	Scales by adding nodes for reads (replicas); writes still serialise on the primary.
Memory (space)	One large coherent memory (e.g. 1-4 TB), simple for the DB cache.	Memory is per node; caches are duplicated on the standby (more total RAM).

TASK 3 - Operating-system structures (4 marks)

Structure	Reliability	Performance	Ease of updating	Example
Simple / monolithic	Low: any driver bug can crash the kernel	Best: services are plain function calls	Poor: rebuild/reboot the whole kernel	Classic UNIX, MS-DOS
Layered	Good: each layer uses only the one below, easy to verify	Worse: each request passes through layers	Moderate	THE system (design idea)
Microkernel	Best: drivers/file system run in user space; a faulty one is restarted	Slower: message passing and context switches	Good: replace servers without reboot	QNX, Mach, MINIX 3
Modular (loadable modules)	Medium: modules run in kernel mode	Near-monolithic	Good: load/unload modules at run time	Linux
Hybrid	Medium-good	Good	Good	Windows NT, macOS/XNU, Android (Linux kernel + userspace services)

- **Linux**: monolithic kernel made modular with loadable kernel modules. **Windows**: hybrid (NT executive in kernel mode, user-mode subsystems). **Android**: Linux kernel + HAL + ART runtime + system services (hybrid layered). **QNX**: true microkernel (scheduler, IPC, interrupts only; everything else is a user process). **FreeRTOS**: tiny kernel linked with the application (scheduler, queues, semaphores; no memory protection unless an MPU port is used).
- **Choice 1 - bedside monitor:** microkernel RTOS (QNX-class). A faulty display or network driver must not stop the alarm task; the driver is restarted in milliseconds and the kernel is small and analysable for timing.
- **Choice 2 - database server:** modular monolithic Linux. Highest I/O and file-system performance (no message passing), loadable modules for storage drivers, and fail-over in the cluster covers kernel crashes.

Alternative: monolithic kernel on the monitors

Criterion	Monolithic	Microkernel (chosen)
Time of one service request	One system call: trap + return, about 0.1-0.3 us. Cost O(1).	User to server message and reply: two IPC transfers with two context switches, typically about 1-5 us (optimised L4-style IPC: under 1 us). Still O(1) but 5-20x larger constant; still far below 10 ms.
Kernel memory	Whole kernel + drivers in kernel space: about 1-10 MB.	Core kernel about 10-100 KB; servers are separate processes (each with its own stack).
Effect of a faulty driver	Corrupts kernel memory, whole monitor crashes (no alarm).	Only that server dies and is restarted; alarm task keeps running.

Values are typical published ranges for this class of system, not measurements from my machine.

TASK 4 - Operating-system services (4 marks)

Service	Monitor	Tablet	Workstation	DB server	Imaging server
User interface	No (small LCD, fixed screens)	Yes (touch)	Yes (GUI)	CLI/remote only	CLI/remote only
Program execution	Yes (fixed set of tasks)	Yes (apps)	Yes	Yes	Yes (job queue)
I/O operations	Yes (sensors, interrupt driven)	Yes	Yes	Yes (high)	Yes (DMA)
File system	Minimal (flash/config)	Yes (app data, encrypted)	Yes	Yes (journaling)	Yes (large files)
Communication	Yes (to central station)	Yes (Wi-Fi)	Yes	Yes	Yes
Error detection	Yes (watchdog, sensor faults)	Yes	Yes	Yes (ECC, fail-over)	Yes
Resource allocation	Simple fixed priorities	Yes (battery aware)	Yes	Yes (memory, I/O)	Yes (CPU, memory)
Accounting	No	Light	Light	Yes (audit)	Yes (usage)
Protection and security	Basic (signed firmware)	Yes (sandbox, encryption, wipe)	Yes	Yes (strong)	Yes

- **Removed from the monitor to make it small and fast:** GUI, multi-user login, general file system, accounting, dynamic program loading, virtual memory (a page fault would add unbounded delay), unused network protocols.
- **Essential for the DB server:** protection and security, file system with journaling, resource allocation, error detection and recovery, communication (cluster heartbeat), accounting/audit.
- **Protecting one patient's records from another user's programs:** user/kernel mode and memory protection keep each process in its own address space; the file system enforces owner/group/ACL permissions on every open(); the database authenticates users and applies role-based access; data are encrypted at rest; every access is logged (accounting) for audit.

Alternative: full desktop OS with every service on the tablets

Criterion	Full desktop OS	Reduced handheld OS (chosen)
Memory/storage (space)	About 2-4 GB RAM idle and 15-30 GB disk for base system and services.	About 1 GB RAM and 4-8 GB storage for base system.
Start-up time	Tens of seconds (many services to start).	A few seconds / instant wake.
Battery life	Shorter: background services keep the CPU awake; may not last a 12 h shift.	Longer: aggressive sleep states, scheduled wake-ups.

TASK 5 - System calls (4 marks)

- **Categories:** process control (fork, exec, wait, exit); file management (open, read, write, close); device management (ioctl, read/write on devices); information maintenance (getpid, time); communication (pipe, socket, shmget); protection (chmod, chown, umask).

What happens when a program calls read(fd, buf, n)

- 1. The program calls the C library wrapper `read()` (an ordinary function call).
- 2. The wrapper puts the system-call number (0 for read on x86-64) in `rax` and the arguments in `rdi, rsi, rdx`.
- 3. It executes the `syscall` trap instruction: hardware switches to kernel mode, saves user PC/flags and jumps to the kernel entry point.
- 4. The kernel looks up the number in the system-call table, validates fd and the user buffer address.
- 5. Kernel work: find the file, copy from the page cache (or start disk I/O, block the process and schedule another), then copy the data to the user buffer.
- 6. The kernel places the return value (bytes read or -errno) in rax and executes `sysret`: mode bit returns to user mode.
- 7. The wrapper returns the value (setting errno on error) to the program.
- **Parameter passing:** (a) in registers (fastest, Linux x86-64: up to 6 arguments); (b) in a block in memory whose address is passed in a register (used when there are many parameters, e.g. Solaris, and for ioctl structures); (c) pushed on the stack by the program and popped by the kernel. Linux uses registers; larger structures are passed by pointer.
- **strace:** `strace -e trace=openat,read,write,close ./copy` prints every system call with arguments and return value; `strace -c ./copy` prints a summary table (calls, errors, time per system call). Expected form: `openat(AT_FDCWD, "records.dat", O_RDONLY) = 3`, then repeated `read(3, "ABC...", 4096) = 4096` and `write(4, ..., 4096) = 4096` lines, ending with a short read and `read(3, "", 4096) = 0`. (Run it yourself and paste the real output.)

Alternative: fopen / fread / fwrite instead of open / read / write

Criterion	Direct system calls	Library (buffered stdio)
System calls for a file of n bytes	n / b calls for a buffer of b bytes (n calls if b = 1): O(n/b).	About n / 4096 (the library reads whole 4 KB blocks even if the program asks for 1 byte): O(n/4096). Example n = 10,000: 3 data reads + 1 end-of-file = 4.
Extra space	Only the program's own buffer.	One extra buffer per open FILE (about 4 KB-8 KB), plus a second copy of the data (kernel to library buffer to user variable).

TASK 6 - Cost of system calls (4 marks)

- **Why a system call costs more than a function call:** a function call is a jump (about 1-2 ns). A system call needs the trap, mode switch, saving registers, argument validation, kernel work, and the return; after it, caches and the TLB may be partly cold (and security mitigations add more). Typical cost is 100-300 ns or more, i.e. 50-100x a function call.
- **Buffer size:** number of read() calls = ceil(n / b) + 1 (the last call returns 0). Smaller buffer, more calls, more overhead.
- **Library buffering:** fread/fwrite fill a 4 KB buffer with one read() and serve later small requests from memory, so 1-byte requests cost about 1 system call per 4096 bytes instead of one per byte.

Complete the Code - TODO 1 (Task06/copy.c)

```
    while (1) {
        n = read(in, buf, bufsize);
        calls++;                        /* count every read call, including the final one */
        if (n <= 0) break;              /* 0 = end of file, -1 = error */
        if (write(out, buf, (size_t)n) != n) { calls = -1; break; }  /* write exactly n bytes */
    }
```

```
Measured output (matches the expected output exactly):
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

Check with `strace -c ./copy`: the "read" row should show the program's total (10001 + 21 + 4 = 10026) plus about 1 extra read made by the dynamic loader when it loads libc (about 10027). Paste your own screenshot. (10,000 / 512 = 19.5, so 20 data reads + 1 end-of-file = 21; 10,000 / 4096 = 2.44, so 3 + 1 = 4.)

Measurements: copying a 64 MB file (Task10/copy_bench.c, measured)

Buffer size	read() calls	Time (ms)	Comment
1 byte	67,108,864	34,054	Dominated by system-call overhead (about 0.5 us per read+write pair)
16 bytes	4,194,304	2,479	16x fewer calls, 14x faster
512 bytes	131,072	216	Still call-bound
4096 bytes	16,384	86.5	One page per call - good
64 KB	1,024	98.4	No further gain (within run-to-run noise)
1 MB	64	54.7	Fastest here; copy cost now dominates
mmap (byte loop)	about 6 calls	176.9	Few system calls, but page faults (n/4096) and a slow byte-by-byte loop in my test

- **Recommendation for the backup program:** use a buffer of 64 KB to 1 MB. Beyond 4 KB the time stops falling quickly because the memory copy and the disk dominate; 64 KB cuts system calls 16-fold compared with 4 KB, stays in cache, and is a safe size on every device. Never use 1-byte or 512-byte buffers (150-400x slower than 4 KB-1 MB).

Alternative: memory-mapped files (mmap)

Criterion	read()/write() with buffer b	mmap
System calls	About n / b (16,384 for 64 MB at b = 4 KB)	A few: open, fstat, mmap, munmap, close (about 6)
Copying kernel to user	Yes: every byte is copied from the page cache to the user buffer	No explicit copy: the page cache pages are mapped into the process (page faults load them, about n/4096 faults)
Memory for very large files	Constant: b bytes, O(b)	Address space O(n); physical pages are loaded on demand and can be evicted, but a 100 GB mapping needs 64-bit address space and may cause paging pressure

TASK 7 - Process-control system calls (4 marks)

- **fork()** creates a child that is a copy of the caller (new PID, copy-on-write address space); returns 0 in the child and the child's PID in the parent. **exec()** replaces the calling process's program with a new one (same PID). **wait()/waitpid()** blocks the parent until a child ends and returns its status. **exit()** terminates the process, closes files and passes the exit status to the parent.
- **How a shell runs a command:** read the line, fork(); child calls exec() on the program; parent calls waitpid(); status = WEXITSTATUS(status) is what shows up as $?.
- **Zombie:** a child that has exited but whose parent has not yet called wait(); only its process-table entry and exit status remain. Cleared when the parent waits (or when the parent dies and init reaps it). **Orphan:** a child whose parent has exited first; it is adopted by init/systemd (PID 1) which reaps it when it ends.
- **Why fork + exec instead of one "create process" call:** between fork and exec the child can change its own environment (redirect stdin/stdout, set up pipes, change directory, drop privileges, set resource limits) using ordinary system calls, so the creation call needs no huge parameter list. Copy-on-write makes the fork itself cheap.

Complete the Code - TODO 1 (Task07/shell.c)

```
    pid_t pid;  int status;                /* declared at the top of main */
    ...
    pid = fork();
    if (pid == 0) {
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);                      /* only reached if exec fails */
    }
    waitpid(pid, &status, 0);
    printf("job 1 exit status %d\n", WEXITSTATUS(status));
```

```
Measured output (identical to the expected output):
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

- **Why stdout is flushed before fork():** when output is a terminal it is line-buffered, but when it goes to a file or a pipe it is fully buffered. Text still sitting in the buffer would be copied into the child by fork(), and both processes would later flush their own copy, printing the text twice.

Experiment (second fflush removed, _exit(3) changed to exit(3), output redirected to a file; Task07/shell_noflush.c). Measured file contents:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2        <- printed by the child's exit(3) flushing the copied buffer
job 1 exit status 0         <- (also copied: everything still unflushed at the fork)
shell: running job 2        <- printed again by the parent at its normal exit
job 2 exit status 3
```

Explanation: with redirection stdout is fully buffered, so "shell: running job 2" (and the earlier "job 1 exit status 0") were still in the buffer at the second fork(). The child inherited a copy. exit(3) runs the C library clean-up and flushes that copy (so the

text appears once from the child); the parent later flushes its own copy (second time). _exit(3) skips the flush, which is why the original program printed it only once.

Process diagram

```
shell (PID 100)  --fork()-->  child (PID 101, copy of shell)
      |                            |  execlp("expr",...)  -> child becomes /usr/bin/expr
      | waitpid(101) (blocked)     |  prints 42, exit(0)
      |<------ exit status 0 --------+  (child is a zombie until the parent waits)
      v  prints "job 1 exit status 0"
```

Alternative: posix_spawn() or threads instead of fork + exec

Criterion	fork + exec	posix_spawn / threads
Time to start 1,000 commands	Measured: 1,000 fork+wait (child exits at once) = about 60 ms (60 us each); with exec add about 0.3-1 ms each (loading the program). O(k) for k commands.	posix_spawn (vfork-like, no page-table copy): usually 2-4x faster for large parents. Threads: measured 1,000 thread create+join = about 12-17 ms (12-17 us each) but threads cannot exec a different program.
Memory copied/shared	Address space copied lazily (copy-on-write): page tables are duplicated, data pages shared until written.	posix_spawn: nothing copied. Threads: everything shared (one address space).
Isolation between commands	Strong: separate address spaces, own descriptors and signals; a crash affects only the child.	posix_spawn: same strong isolation. Threads: none - a bug in one command can corrupt or kill the whole shell.

TASK 8 - CPU states and protection (3 marks)

- **User mode and kernel mode:** a mode bit in the CPU status register (0 = kernel, 1 = user). Application code runs in user mode and cannot execute **privileged instructions** (disable interrupts, load page-table base, halt, I/O instructions, change the mode bit). A system call (trap) switches to kernel mode and jumps to a fixed kernel address; returning sets user mode again.
- **Interrupt:** asynchronous signal from hardware (sensor, disk, timer); **trap/exception:** synchronous, caused by the running instruction (system call, divide by zero, page fault). Both save the state, switch to kernel mode and run a handler found through the interrupt vector. The **timer interrupt** returns control to the OS every few milliseconds, so no program can keep the CPU forever (pre-emption, time slicing).
- **If any program could disable interrupts:** a faulty or malicious program could stop the sensor interrupts and the timer, so the monitor would never see an abnormal reading, the scheduler could never pre-empt it, and the alarm would be lost. Hence disabling interrupts is privileged.
- **Memory protection:** base and limit registers (or the page table with valid/permission bits, checked by the MMU on every access). Each process gets its own page table; an access outside its pages causes a trap and the OS terminates the process (segmentation fault). Changing the page-table base register is privileged.

Alternative: polling instead of interrupts on the monitors

Criterion	Polling (loop reads the sensor)	Interrupt-driven (chosen)
Worst-case delay before an abnormal reading is noticed	One polling interval T plus processing: to meet 10 ms with 2 ms processing, T must be at most 8 ms (poll at 125 Hz or faster).	Interrupt latency: a few microseconds (plus the handler), independent of how rarely the event occurs.
CPU time and power	The CPU wakes every T whether or not anything happened: 1 kHz polling at 2 us each = 0.2 % of CPU, but the CPU can never stay in deep sleep. Wasted work O(1/T).	CPU sleeps until the sensor interrupts; work only on real events.

TASK 9 - Memory hierarchy and I/O systems (4 marks)

Level	Typical access time	Typical size	Used in the hospital for
Registers	about 0.3 ns	Under 1 KB	Variables of the alarm-checking code
Cache (L1/L2/L3)	1-15 ns	32 KB - tens of MB	Hot code, current sensor window
Main memory (DRAM)	about 80-100 ns	GB - TB	Database buffer pool, image being processed, last minutes of monitor readings
SSD	about 20-100 us	0.5-10 TB	Active patient records, indexes, recent MRI studies, journal/log
Hard disk	about 5-10 ms	4-20 TB per disk	Older MRI archives, backups
Network/cloud storage	about 1-100 ms	PB	Archive, off-site backup, records shared with partners

- **Placement plan:** monitor readings - RAM ring buffer, copied to the central DB asynchronously; patient records - DB on SSD with hot rows cached in RAM, replicated to the standby; MRI images - SSD while being processed and for recent studies, then hard-disk/network archive; backups - hard disk and off-site.
- **I/O methods:** programmed I/O (CPU loops waiting for the device - wastes CPU, only for tiny devices); interrupt-driven I/O (device interrupts per unit of data - good for sensors/keyboards); DMA (controller moves whole blocks to memory, one interrupt at the end - best for the MRI scanner and disks).

- **Effect of caching on record look-up:** effective access time = h x t_{cache} + (1-h) x t_{miss}. Example: t_{cache} = 100 ns, t_{SSD} = 100 us: at 90 % hit ratio, 10.1 us; at 99 %, 1.1 us - a 9x gain from the last 9 % of hits.

Alternative: interrupt-driven I/O (one interrupt per word) for a 500 MB image

Criterion	Interrupt per 4-byte word	DMA (64 KB blocks, chosen)
Number of interrupts	500 MB = 524,288,000 B / 4 B = 131,072,000 interrupts, O(n).	524,288,000 / 65,536 = 8,000 interrupts, O(n/block).
CPU time (assume 2 us per interrupt)	131,072,000 x 2 us = about 262 s of CPU time - the server could do nothing else.	8,000 x 2 us = 16 ms, and the data copy is done by the DMA controller.
Buffers needed	One word register/small FIFO; CPU copies each word to memory.	A 64 KB (or larger, double-buffered) buffer in memory, or the whole image buffer.

TASK 10 - Measure and compare (4 marks)

(a) System calls versus buffer size (strace -c and the program counts)

Buffer	read() calls with data (64 MB file, measured)	Predicted n / b	Time (ms, measured)
1	67,108,864	67,108,864	34,054
512	131,072	131,072	216
4096	16,384	16,384	86.5
65,536	1,024	1,024	98.4
1,048,576	64	64	54.7

The counts equal n/b exactly (the benchmark counts only reads that returned data; the Task 6 program also counts the final end-of-file read, giving +1). Confirm with `strace -c`. Times are from a single run and are noisy; the trend (calls fall 1/b, time falls steeply until about 4 KB, then flattens) is what matters.

(b) Time in user mode and kernel mode (bash "time ./forkthread", measured)

```
1000 fork+wait   :    60.36 ms (60.4 us each)
1000 thread+join :    11.85 ms (11.9 us each)
ratio fork/thread: 5.1x
real 0m0.073s   user 0m0.028s   sys 0m0.039s
```

Most of the time is **sys** (kernel mode): process creation is kernel work (copying page tables, allocating a PCB). For the byte-by-byte copy, sys time dominated user time for the same reason: every byte cost a trap.

(c) Creating 1,000 processes vs 1,000 threads

	1,000 processes (fork)	1,000 threads (pthread_create)
Creation + join time (measured, two runs)	59-60 ms	12-17 ms (3.6-5.1x faster)
Memory per unit	Own page tables and PCB (about 10-20 KB kernel memory) + private copy of any page written	Own stack (8 MB virtual, only touched pages resident) + small thread control block
What is shared	Nothing by default (COW pages, then private)	Code, heap, globals, open files; only stack and registers are private

- **Support or contradict earlier judgements:** measurements support the conclusions of Tasks 5-7: system calls dominate when buffers are tiny (so buffered I/O or large buffers are the right design), threads are cheaper than processes but give no isolation (so the isolation-critical parts stay separate processes/machines), and the strong 5x gap confirms that a "process per request" design is expensive. They do not measure real-time latency; that claim (RTOS needed for the monitor) rests on worst-case analysis, which an ordinary Linux benchmark cannot prove.

TASK 11 - Recommended operating-system plan (4 marks)

Environment	OS type / structure	Services	Protection	I/O method
Bedside monitor	Hard RTOS, microkernel (QNX-class)	Scheduler, interrupts, minimal comms; no GUI/files	Signed firmware, MMU isolation of tasks, watchdog	Interrupt-driven sensors; DMA to network
Nurse tablet	Handheld OS (Android-class) with device management	Apps, secure Wi-Fi, power management	App sandbox, disk encryption, remote wipe, screen lock	Interrupt-driven; DMA for storage
Doctor workstation	Desktop time-sharing OS	GUI, file system, network	User accounts, ACLs, auditing	Interrupt + DMA
Database cluster	Linux server, modular monolithic, 2-3 node fail-over cluster	Journaling file system, resource control, heartbeat	Roles/ACLs, encryption, audit log	DMA, large caches
Imaging server	Linux SMP multiprocessor with job queue	Batch scheduling, large memory, NUMA aware	Per-job isolation (containers), encrypted storage	DMA from scanner and disks
Partner link	Distributed layer on server OS	Replication, message queue	VPN/TLS, mutual authentication, audit	Interrupt + DMA NIC

- **Limitations and risks:** five platforms mean more skills and patch processes; the RTOS and tablet OS have fewer security tools; microkernel IPC cost; cluster split-brain risk; shared storage still a single point of failure unless replicated; staff must manage certificates and keys for the partner link.
- **Three hospitals:** the per-ward and per-hospital design scales (monitors and tablets scale by number of devices, O(devices)). The DB must move from a local cluster to a replicated, geographically distributed database (consistency vs latency trade-off), and identity management should be federated across hospitals.

Alternative: virtual machines on two large servers

Criterion	VMs on two servers	Separate physical machines (chosen)
Monitor response time	Hypervisor scheduling adds jitter (tens of us to ms) - unsafe for the 10 ms hard deadline unless an RT hypervisor with dedicated cores is used. Monitors should stay on their own hardware; only servers/workstation back-ends can be virtualised.	Direct hardware, bounded latency.
Memory and CPU overhead	Each VM has a full guest OS (0.5-2 GB) and about 2-10 % CPU virtualisation overhead; I/O overhead higher without paravirtual drivers.	No overhead but under-utilised hardware.
Reliability	Two hosts: live migration and snapshots help, but a host failure takes many VMs at once - a big failure domain.	Failures are isolated per machine; more hardware to fail.

TASK 12 - Re-evaluate and adapt the design (8 marks)

Changed requirement: home monitoring devices send readings over the Internet; patient records move to a cloud service shared with partner hospitals.

A. Impact analysis (2 marks)

Area	Affected?	Impact
Monitors / ward RTOS	Not affected	Local alarm path unchanged.
New home devices	New environment	Need a small embedded RTOS or hardened Linux, battery life, remote firmware updates, unreliable home networks.
Database cluster	Affected	Becomes (or feeds) a cloud record service: distributed system, replication, consistency.
Partner link	Redesigned	Replaced by the shared cloud service with identity federation.
Services	Affected	Communication, remote update, error detection (offline buffering), accounting/audit of cross-organisation access.
Protection	Strongly affected	Attack surface is now the Internet: authentication of devices, TLS, key management, tenant isolation in the cloud.

B. Evaluation of the original design (2 marks)

- **Response time:** a home alarm cannot depend on a round trip to the cloud (Internet delay 20-200 ms, can fail), so the 10 ms alarm must be raised locally on the device; the original ward design already does this. The original OS plan has no OS for the home device.
- **Reliability:** the on-site cluster covers hardware failures, but not an Internet outage or a cloud outage; home devices need offline buffering and retry.
- **Security:** the original plan assumed a trusted hospital network and a small number of partner links. Thousands of home devices and a shared cloud require device identity, encrypted transport, least privilege per hospital and audit - not supported by the original plan.
- **Verdict:** the ward design can be **retained** for monitors, tablets, workstations and imaging; the records/partner part must be **modified/redesigned**.

C. Adaptation (2 marks)

- **Home device:** embedded RTOS (FreeRTOS/Zephyr class) or minimal hardened Linux, with local threshold alarm, secure boot, signed over-the-air updates, TLS 1.3 with device certificates, store-and-forward buffer, sleep modes for battery.
- **Ingestion tier:** message broker (e.g. MQTT) behind a gateway, running in containers on a server OS with autoscaling.
- **Records:** distributed, replicated database in the cloud (3 replicas, 2 regions) with strong consistency for medication/allergy data and eventual consistency for non-critical data; the on-site cluster stays as a local cache/fallback, so the wards still work if the Internet is down.
- **Protection:** federated identity, role-based access, encryption at rest and in transit, per-hospital access logs.

D. Evidence-based defence (2 marks)

- **Decision: MODIFY** (retain the ward design, redesign the records/sharing layer, add a new home-device class).
- **Evidence:** Task 8/9 analysis and Task 10 measurements show hard latency must be bounded locally (interrupt latency in microseconds versus Internet delay in tens of milliseconds); Task 6 showed that fewer, larger transfers reduce overhead, which supports batching home readings; Task 11 limits show VMs/cloud add jitter and are unsuitable for the alarm path.

- **Trade-offs:** cloud gives sharing, scaling and off-site resilience but adds latency, dependence on the Internet and a bigger security surface; keeping the local cluster costs extra money and duplicate data but preserves availability when the link is down. A complete redesign would throw away a working, measured ward design without gaining anything for the alarm path.